

**Lab Task:- Mutation Lab Testing**  
**Course: SE-605**

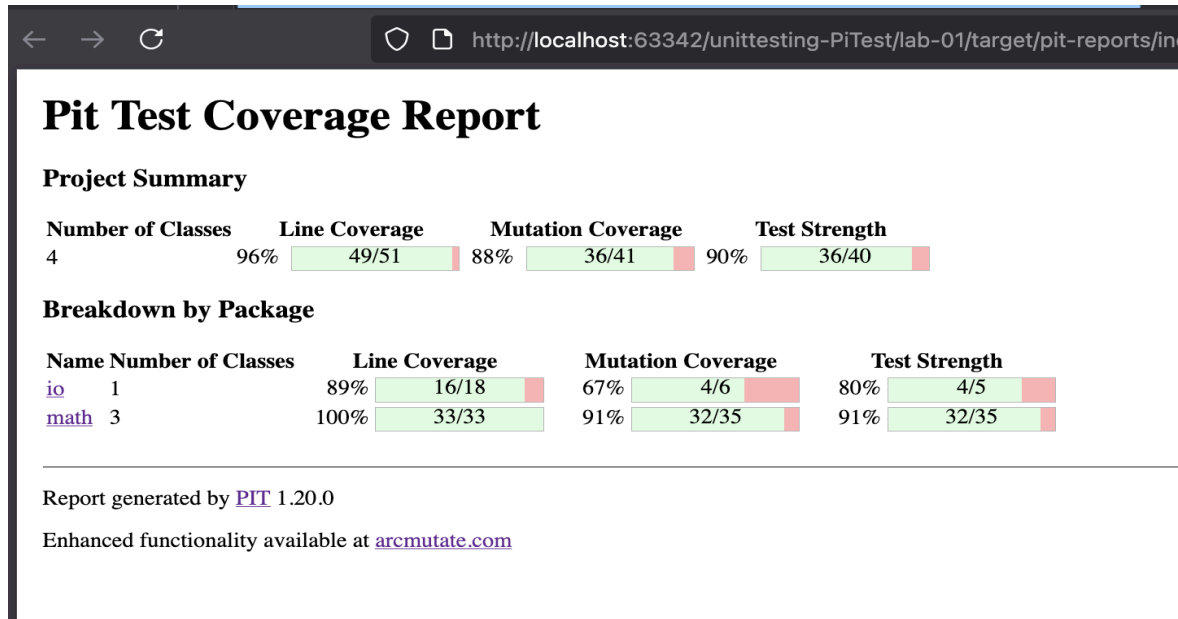
**Submitted by:**  
**Md. Rakibul Islam**  
**Roll: 1411**

**Submitted to:**  
**Mridha Md. Nafis Fuad**  
**Lecture, IIT**  
**University of Dhaka**

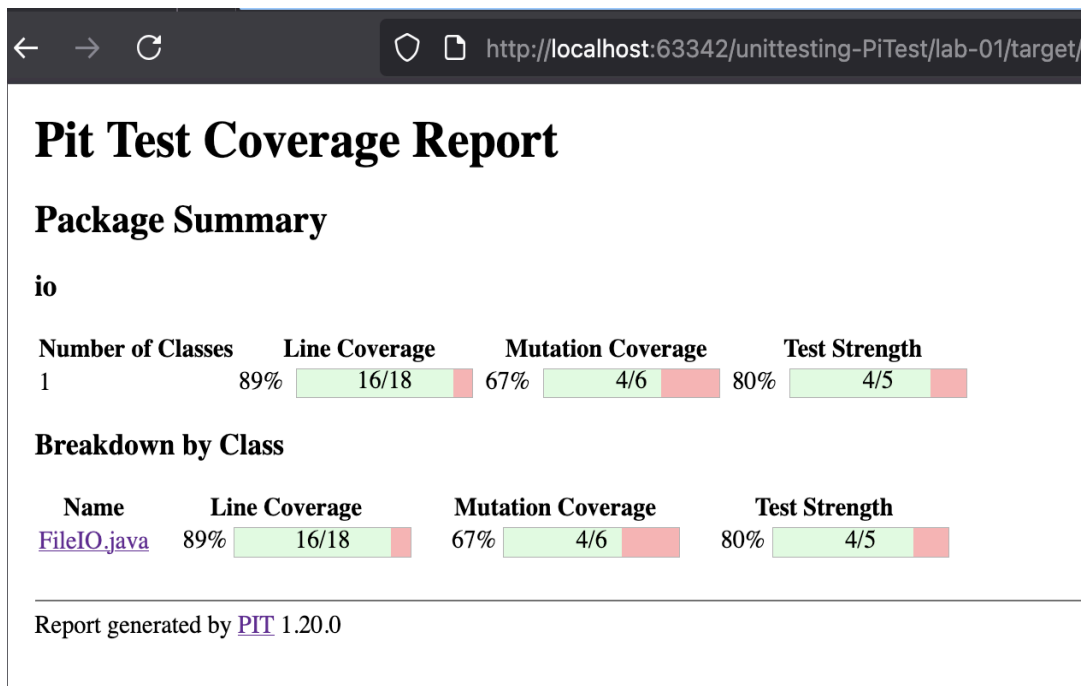
**Submission date: 10-07-2025**

# Step-1: Existing test cases result

## 1. Overall test result



## 2. For **io** package details test result and coverage



### 2.1 FileIO.java class coverage

## FileIO.java

```
1  package io;
2
3  import java.io.BufferedReader;
4  import java.io.File;
5  import java.io.FileNotFoundException;
6  import java.io.FileReader;
7  import java.io.IOException;
8  import java.util.ArrayList;
9  import java.util.List;
10
11  /**
12   * The FileIO provides simple file input/output operations that serve as
13   * hands-on practice on Unit Testing.
14   *
15   * @author agkortzis
16   * @version 1.0
17   * @since 2020-04-06
18   */
19  public class FileIO {
20
21      /**
22       * Reads a file that contains numbers line by line
23       * and returns an array of the integers found in the file.
24       * @param filepath the file that contains the numberses
25       * @return an array of numbers
26       * @exception IllegalArgumentException when the given file does not exist
27       * @exception IllegalArgumentException when the given file is empty
28       * @exception NumberFormatException for checking invalid entries
29       * @exception IOException when an IO interruption occurs (not required to be tested)
30       */
31      public int[] readFile(String filepath) {
32          File file = new File(filepath);
33          1 if (!file.exists())
34              throw new IllegalArgumentException("Input file does not exist");
35
36          List<Integer> numbersList = new ArrayList<>();
37          BufferedReader reader;
38          try {
39              reader = new BufferedReader(new FileReader(file));
40              String line = null;
41              1 while ((line = reader.readLine()) != null) {
42                  try {
43                      int number = Integer.parseInt(line);
44                      numbersList.add(number);
45                  } catch (NumberFormatException e) {
46                      // Do nothing will skip the the current invalid line
47                  }
48              }
49          } catch (IOException e) {
50              1 e.printStackTrace();
51          }
52
53          1 if (numbersList.size() == 0)
54              throw new IllegalArgumentException("Given file is empty");
55
56          // Convert a List to an array using
57          2 return numbersList.stream().mapToInt(i -> i).toArray();
58      }
59
60  }
```

## Mutations

|                    |                                                                                                                                                  |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">33</a> | 1. removed conditional - replaced equality check with false → SURVIVED <a href="#">Covering tests</a>                                            |
| <a href="#">41</a> | 1. removed conditional - replaced equality check with false → KILLED                                                                             |
| <a href="#">50</a> | 1. removed call to java/io/IOException::printStackTrace → NO_COVERAGE                                                                            |
| <a href="#">53</a> | 1. removed conditional - replaced equality check with false → KILLED                                                                             |
| <a href="#">57</a> | 1. replaced int return with 0 for io/FileIO::lambda\$readFile\$0 → KILLED<br>2. replaced return value with null for io/FileIO::readFile → KILLED |

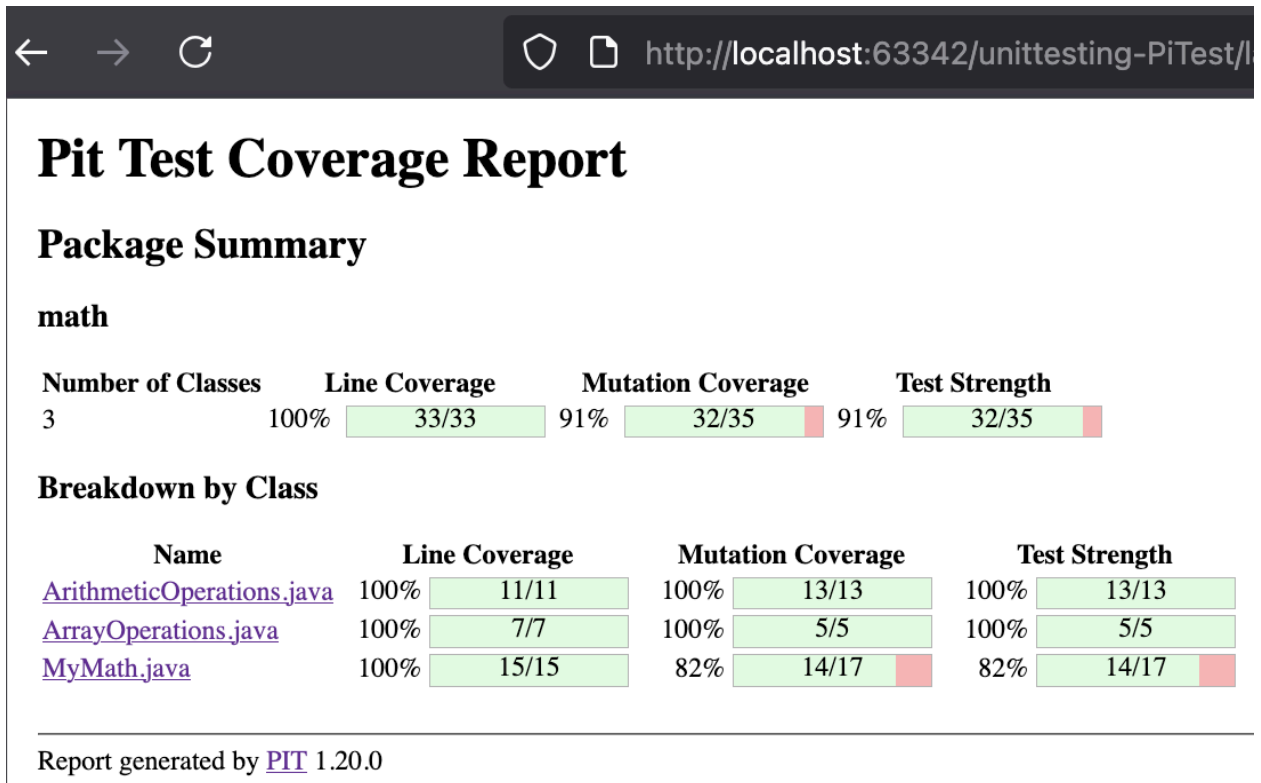
## Active mutators

- CONDITIONALS\_BOUNDARY
- EMPTY\_RETURNS
- FALSE\_RETURNS
- INCREMENTS
- INVERT\_NEGS
- MATH
- NULL\_RETURNS
- PRIMITIVE\_RETURNS
- REMOVE\_CONDITIONALS\_EQUAL\_ELSE
- REMOVE\_CONDITIONALS\_ORDER\_ELSE
- TRUE\_RETURNS
- VOID\_METHOD\_CALLS

## Tests examined

- io.FileIOTest.[engine:junit-jupiter]/[class:io.FileIOTest]/[method:testFileDoesNotExist\_ThrowsException()] (1 ms)
- io.FileIOTest.[engine:junit-jupiter]/[class:io.FileIOTest]/[method:testValidFile\_AllValidLines()] (1 ms)
- io.FileIOTest.[engine:junit-jupiter]/[class:io.FileIOTest]/[method:testEmptyFile\_ThrowsException()] (3 ms)
- io.FileIOTest.[engine:junit-jupiter]/[class:io.FileIOTest]/[method:testInvalidFile\_MixedValidAndInvalidLines()] (17 ms)
- math.ArrayOperationsTest.[engine:junit-jupiter]/[class:math.ArrayOperationsTest]/[method:findPrimesInFile()] (3 ms)

### 3. For **math** package details test result and coverage



## 3.1 ArithmeticOperations.java class coverage

← → ↻ <http://localhost:63342/unittesting-PiTest/lab-01/target/pit-reports/math/ArithmeticOperations.java>

### ArithmeticOperations.java

```
1 package math;
2
3 /**
4  * The ArithmeticOperations provides simple arithmetic operations that serve as
5  * hands-on practice on Unit Testing.
6  *
7  * @author agkortzis
8  * @version 1.0
9  * @since 2020-04-06
10 */
11 public class ArithmeticOperations {
12
13     /**
14      * Performs the basic arithmetic operation of division.
15      *
16      * @param numerator the numerator of the operation
17      * @param denominator the denominator of the operation
18      * @return the result of the division between numerator and denominator
19      * @exception ArithmeticException when the denominator is zero
20      */
21     public double divide(double numerator, double denominator) {
22         if (denominator == 0)
23             throw new ArithmeticException("Cannot divide with zero");
24
25         return numerator / denominator;
26     }
27
28     /**
29      * Performs the basic arithmetic operation of multiplication between two
30      * positive Integers
31      *
32      * @param x the first input
33      * @param y the second input
34      * @return the product of the multiplication
35      * @exception IllegalArgumentException when <b>x</b> or <b>y</b> are negative
36      *         numbers
37      * @exception IllegalArgumentException when the product does not fit in an
38      *         Integer variable
39      */
40     public int multiply(int x, int y) {
41         if (x < 0 || y < 0) {
42             throw new IllegalArgumentException("x & y should be >= 0");
43         } else if (y == 0) {
44             return 0;
45         } else if (x <= Integer.MAX_VALUE / y) {
46             return x * y;
47         } else {
48             throw new IllegalArgumentException("The product does not fit in an Integer variable");
49         }
50     }
51
52 }
```

## Mutations

|    |                                                                                                                                                                                                                                          |
|----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 22 | 1. removed conditional - replaced equality check with false → KILLED                                                                                                                                                                     |
| 25 | 1. Replaced double division with multiplication → KILLED<br>2. replaced double return with 0.0d for math/ArithmeticOperations::divide → KILLED                                                                                           |
| 41 | 1. removed conditional - replaced comparison check with false → KILLED<br>2. removed conditional - replaced comparison check with false → KILLED<br>3. changed conditional boundary → KILLED<br>4. changed conditional boundary → KILLED |
| 43 | 1. removed conditional - replaced equality check with false → KILLED                                                                                                                                                                     |
| 45 | 1. Replaced integer division with multiplication → KILLED<br>2. changed conditional boundary → KILLED<br>3. removed conditional - replaced comparison check with false → KILLED                                                          |
| 46 | 1. Replaced integer multiplication with division → KILLED<br>2. replaced int return with 0 for math/ArithmeticOperations::multiply → KILLED                                                                                              |

## Active mutators

- CONDITIONALS\_BOUNDARY
- EMPTY\_RETURNS
- FALSE\_RETURNS
- INCREMENTS
- INVERT\_NEGS
- MATH
- NULL\_RETURNS
- PRIMITIVE\_RETURNS
- REMOVE\_CONDITIONALS\_EQUAL\_ELSE
- REMOVE\_CONDITIONALS\_ORDER\_ELSE
- TRUE\_RETURNS
- VOID\_METHOD\_CALLS

## Tests examined

- math.ArithmeticOperationsTest.[engine:junit-jupiter]/[class:math.ArithmeticOperationsTest]/[method:divide\_small\_numbers()] (0 ms)
- math.ArithmeticOperationsTest.[engine:junit-jupiter]/[class:math.ArithmeticOperationsTest]/[method:divide\_both\_negative\_numbers()] (0 ms)
- math.ArithmeticOperationsTest.[engine:junit-jupiter]/[class:math.ArithmeticOperationsTest]/[method:divide\_numerator\_zero()] (0 ms)
- math.ArithmeticOperationsTest.[engine:junit-jupiter]/[class:math.ArithmeticOperationsTest]/[method:divide\_by\_one()] (0 ms)
- math.ArithmeticOperationsTest.[engine:junit-jupiter]/[class:math.ArithmeticOperationsTest]/[method:divide\_fractional\_numbers()] (0 ms)
- math.ArithmeticOperationsTest.[engine:junit-jupiter]/[class:math.ArithmeticOperationsTest]/[method:divide\_positive\_numbers()] (0 ms)
- math.ArithmeticOperationsTest.[engine:junit-jupiter]/[class:math.ArithmeticOperationsTest]/[method:divide\_by\_negative\_one()] (0 ms)
- math.ArithmeticOperationsTest.[engine:junit-jupiter]/[class:math.ArithmeticOperationsTest]/[method:divide\_positive\_numerator\_negative\_denominator()] (0 ms)
- math.ArithmeticOperationsTest.[engine:junit-jupiter]/[class:math.ArithmeticOperationsTest]/[method:divide\_negative\_numerator\_positive\_denominator()] (0 ms)
- math.ArithmeticOperationsTest.[engine:junit-jupiter]/[class:math.ArithmeticOperationsTest]/[method:divide\_large\_numbers()] (2 ms)
- math.ArithmeticOperationsTest.[engine:junit-jupiter]/[class:math.ArithmeticOperationsTest]/[method:divideDenominatorZero()] (1 ms)
- math.ArithmeticOperationsTest.[engine:junit-jupiter]/[class:math.ArithmeticOperationsTest]/[method:multiply\_boundarySquareNumbers()] (0 ms)
- math.ArithmeticOperationsTest.[engine:junit-jupiter]/[class:math.ArithmeticOperationsTest]/[method:multiply\_oneAndMaxValue()] (0 ms)
- math.ArithmeticOperationsTest.[engine:junit-jupiter]/[class:math.ArithmeticOperationsTest]/[method:multiply\_zeroX\_positiveY()] (0 ms)
- math.ArithmeticOperationsTest.[engine:junit-jupiter]/[class:math.ArithmeticOperationsTest]/[method:multiply\_smallNumbers()] (1 ms)
- math.ArithmeticOperationsTest.[engine:junit-jupiter]/[class:math.ArithmeticOperationsTest]/[method:multiply\_xTooLargeForY\_throwsOverflowException()] (0 ms)
- math.ArithmeticOperationsTest.[engine:junit-jupiter]/[class:math.ArithmeticOperationsTest]/[method:multiply\_yTooLargeForX\_throwsOverflowException()] (0 ms)
- math.ArithmeticOperationsTest.[engine:junit-jupiter]/[class:math.ArithmeticOperationsTest]/[method:multiply\_positiveX\_zeroY()] (1 ms)
- math.ArithmeticOperationsTest.[engine:junit-jupiter]/[class:math.ArithmeticOperationsTest]/[method:multiply\_zeroXZeroY()] (2 ms)
- math.ArithmeticOperationsTest.[engine:junit-jupiter]/[class:math.ArithmeticOperationsTest]/[method:multiply\_negativeY\_throwsIllegalArgumentException()] (1 ms)
- math.ArithmeticOperationsTest.[engine:junit-jupiter]/[class:math.ArithmeticOperationsTest]/[method:multiply\_bothNegative\_throwsIllegalArgumentException()] (0 ms)
- math.ArithmeticOperationsTest.[engine:junit-jupiter]/[class:math.ArithmeticOperationsTest]/[method:multiply\_negativeX\_throwsIllegalArgumentException()] (1 ms)

## 3.2 ArrayOrepations.java class coverage

```
← → ↻ http://localhost:63342/unittesting-PiTest/lab-01/target/pit-reports/

ArrayOperations.java

1  package math;
2
3  import java.util.ArrayList;
4  import java.util.Arrays;
5  import java.util.List;
6
7  import io.FileIO;
8
9  /**
10   * The MyMath provides simple methods such as computing a factorial or finding
11   * whether an integer is prime
12   *
13   * @author pandeliskirpoglou
14   * @version 1.0
15   * @since 2020-04-18
16   */
17
18  public class ArrayOperations {
19
20      /**
21       * Gets one integer and returns true if it is prime and false if it is not.
22       *
23       * @param fileio    instance for reading a file
24       * @param filepath  path for a file that needs to be checked
25       * @param myMath    instance for checking whether a number is prime
26       * @return arrayOfPrimeNumbers the array of prime numbers that where in the file
27       */
28
29      public int[] findPrimesInFile(FileIO fileio, String filepath, MyMath myMath) {
30          int[] arrayOfNumbers = fileio.readFile(filepath);
31          List<Integer> arrayOfPrimeNumbers = new ArrayList<>();
32          for (int i = 0; i < arrayOfNumbers.length; i++) {
33              if (myMath.isPrime(arrayOfNumbers[i])) {
34                  arrayOfPrimeNumbers.add(arrayOfNumbers[i]);
35              }
36          }
37          return arrayOfPrimeNumbers.stream().mapToInt(i -> i).toArray();
38      }
39
40  }
```



## Mutations

|                    |                                                                                                                                                                                        |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">32</a> | 1. removed conditional - replaced comparison check with false → KILLED<br>2. changed conditional boundary → KILLED                                                                     |
| <a href="#">33</a> | 1. removed conditional - replaced equality check with false → KILLED                                                                                                                   |
| <a href="#">37</a> | 1. replaced int return with 0 for math/ArrayOperations::lambda\$findPrimesInFile\$0 → KILLED<br>2. replaced return value with null for math/ArrayOperations::findPrimesInFile → KILLED |

## Active mutators

- CONDITIONALS\_BOUNDARY
- EMPTY\_RETURNS
- FALSE\_RETURNS
- INCREMENTS
- INVERT\_NEGS
- MATH
- NULL\_RETURNS
- PRIMITIVE\_RETURNS
- REMOVE\_CONDITIONALS\_EQUAL\_ELSE
- REMOVE\_CONDITIONALS\_ORDER\_ELSE
- TRUE\_RETURNS
- VOID\_METHOD\_CALLS

## Tests examined

- math.ArrayOperationsTest.[engine:junit-jupiter]/[class:math.ArrayOperationsTest]/[method:findPrimesInFile()] (3 ms)

## 3.3 MyMath.java class coverage

← → ↺ <http://localhost:63342/unittesting-PITest/lab-01/target/pit-reports/math/MyMath.java>

### MyMath.java

```
1 package math;
2
3 /**
4  * The MyMath provides simple methods such as computing a factorial or finding
5  * whether an integer is prime
6  *
7  * @author pandeliskirpoglou
8  * @version 1.0
9  * @since 2020-04-15
10 */
11
12 public class MyMath {
13
14     /**
15      * Gets one integer and returns it's factorial.
16      *
17      * @param n the number of which we want the factorial
18      * @return fact the factorial of the number n
19      * @exception IllegalArgumentException when inputs n < 0 and n > 12
20      */
21     public int factorial(int n) {
22         int fact = 1;
23         if (n < 0 || n > 12) {
24             throw new IllegalArgumentException("number should be 0 or above and 12 or below");
25         } else {
26             for (int i = 1; i <= n; i++) {
27                 fact = fact * i;
28             }
29         }
30     }
31     return fact;
32 }
33
34 /**
35  * Gets one integer and returns true if it is prime and false if it is not.
36  *
37  * @param n the number we are trying to find out whether it is prime or not
38  * @return isPrimeNumber true if n is prime | false if n is not prime
39  * @exception IllegalArgumentException when inputs n < 2
40      */
41     public boolean isPrime(int n) {
42         boolean isPrimeNumber = true;
43         if (n < 2) {
44             throw new IllegalArgumentException("No prime numbers below 2");
45         } else {
46             for (int i = 2; i <= n / 2; ++i) { // Checking to n/2 for complexity
47                 if (n % i == 0) {
48                     isPrimeNumber = false;
49                     break;
50                 }
51             }
52         }
53     }
54     return isPrimeNumber;
55 }
56 }
57
58
59 }
```

## Mutations

1. removed conditional - replaced comparison check with false → KILLED  
2. removed conditional - replaced comparison check with false → KILLED  
3. changed conditional boundary → SURVIVED [Covering tests](#)  
Covered by tests:

- math.MyMathTest.[engine:junit-jupiter]/[class:math.MyMathTest]/[method:factorialOutsideRange()]
- math.MyMathTest.[engine:junit-jupiter]/[class:math.MyMathTest]/[method:factorialWithinRange()]

24

4. changed conditional boundary → SURVIVED [Covering tests](#)  
Covered by tests:

- math.MyMathTest.[engine:junit-jupiter]/[class:math.MyMathTest]/[method:factorialOutsideRange()]
- math.MyMathTest.[engine:junit-jupiter]/[class:math.MyMathTest]/[method:factorialWithinRange()]

27

1. changed conditional boundary → KILLED  
2. removed conditional - replaced comparison check with false → KILLED

28

1. Replaced integer multiplication with division → KILLED

32

1. replaced int return with 0 for math/MyMath::factorial → KILLED

45

1. changed conditional boundary → KILLED  
2. removed conditional - replaced comparison check with false → KILLED

1. removed conditional - replaced comparison check with false → KILLED  
2. changed conditional boundary → SURVIVED [Covering tests](#)  
Covered by tests:

48

- math.MyMathTest.[engine:junit-jupiter]/[class:math.MyMathTest]/[method:isPrime()]
- math.ArrayOperationsTest.[engine:junit-jupiter]/[class:math.ArrayOperationsTest]/[method:findPrimesInFile()]

3. Replaced integer division with multiplication → KILLED

49

1. Replaced integer modulus with multiplication → KILLED  
2. removed conditional - replaced equality check with false → KILLED

56

1. replaced boolean return with true for math/MyMath::isPrime → KILLED  
2. replaced boolean return with false for math/MyMath::isPrime → KILLED

## Active mutators

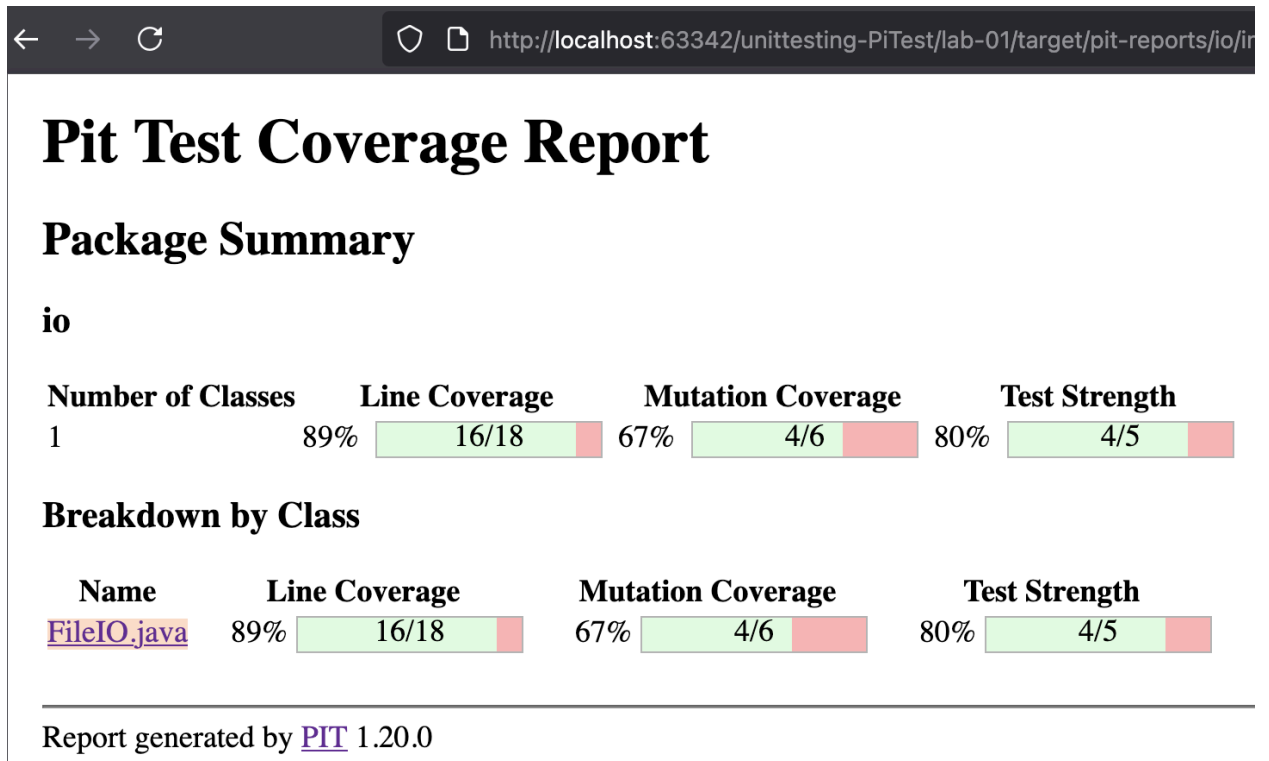
- CONDITIONALS\_BOUNDARY
- EMPTY\_RETURNS
- FALSE\_RETURNS
- INCREMENTS
- INVERT\_NEGS
- MATH
- NULL\_RETURNS
- PRIMITIVE\_RETURNS
- REMOVE\_CONDITIONALS\_EQUAL\_ELSE
- REMOVE\_CONDITIONALS\_ORDER\_ELSE
- TRUE\_RETURNS
- VOID\_METHOD\_CALLS

## Tests examined

- math.MyMathTest.[engine:junit-jupiter]/[class:math.MyMathTest]/[method:isPrime()] (0 ms)
- math.ArrayOperationsTest.[engine:junit-jupiter]/[class:math.ArrayOperationsTest]/[method:findPrimesInFile()] (3 ms)
- math.MyMathTest.[engine:junit-jupiter]/[class:math.MyMathTest]/[method:factorialWithinRange()] (0 ms)
- math.MyMathTest.[engine:junit-jupiter]/[class:math.MyMathTest]/[method:factorialOutsideRange()] (0 ms)
- math.MyMathTest.[engine:junit-jupiter]/[class:math.MyMathTest]/[method:isPrimeLessThanTwo()] (1 ms)

## Step-2: A detailed explanation of why certain classes scored poorly reveals the causes of their low performance.

### 1. For io package, FileIO.java class shows the low score



## FileIO.java

```
1  package io;
2
3  import java.io.BufferedReader;
4  import java.io.File;
5  import java.io.FileNotFoundException;
6  import java.io.FileReader;
7  import java.io.IOException;
8  import java.util.ArrayList;
9  import java.util.List;
10
11  /**
12   * The FileIO provides simple file input/output operations that serve as
13   * hands-on practice on Unit Testing.
14   *
15   * @author agkortzis
16   * @version 1.0
17   * @since 2020-04-06
18   */
19  public class FileIO {
20
21      /**
22       * Reads a file that contains numbers line by line
23       * and returns an array of the integers found in the file.
24       * @param filepath the file that contains the numberses
25       * @return an array of numbers
26       * @exception IllegalArgumentException when the given file does not exist
27       * @exception IllegalArgumentException when the given file is empty
28       * @exception NumberFormatException for checking invalid entries
29       * @exception IOException when an IO interruption occurs (not required to be tested)
30       */
31      public int[] readFile(String filepath) {
32          File file = new File(filepath);
33          if (!file.exists())
34              throw new IllegalArgumentException("Input file does not exist");
35
36          List<Integer> numbersList = new ArrayList<>();
37          BufferedReader reader;
38          try {
39              reader = new BufferedReader(new FileReader(file));
40              String line = null;
41              while ((line = reader.readLine()) != null) {
42                  try {
43                      int number = Integer.parseInt(line);
44                      numbersList.add(number);
45                  } catch (NumberFormatException e) {
46                      // Do nothing will skip the the current invalid line
47                  }
48              }
49          } catch (IOException e) {
50              e.printStackTrace();
51          }
52
53          if (numbersList.size() == 0)
54              throw new IllegalArgumentException("Given file is empty");
55
56          // Convert a List to an array using
57          return numbersList.stream().mapToInt(i -> i).toArray();
58      }
59
60  }
```

## Mutations

1. removed conditional - replaced equality check with false → SURVIVED [Covering tests](#)  
Covered by tests:

33

- io.FileIOTest.[engine:junit-jupiter]/[class:io.FileIOTest]/[method:testFileDoesNotExist\_ThrowsException()]
- io.FileIOTest.[engine:junit-jupiter]/[class:io.FileIOTest]/[method:testValidFile\_AllValidLines()]
- io.FileIOTest.[engine:junit-jupiter]/[class:io.FileIOTest]/[method:testEmptyFile\_ThrowsException()]
- io.FileIOTest.[engine:junit-jupiter]/[class:io.FileIOTest]/[method:testInvalidFile\_MixedValidAndInvalidLines()]
- math.ArrayOperationsTest.[engine:junit-jupiter]/[class:math.ArrayOperationsTest]/[method:findPrimesInFile()]

41 1. removed conditional - replaced equality check with false → KILLED

50 1. removed call to java/io/IOException::printStackTrace → NO\_COVERAGE

53 1. removed conditional - replaced equality check with false → KILLED

57 1. replaced int return with 0 for io/FileIO::lambda\$readFile\$0 → KILLED  
2. replaced return value with null for io/FileIO::readFile → KILLED

## Active mutators

- CONDITIONALS\_BOUNDARY
- EMPTY\_RETURNS
- FALSE\_RETURNS
- INCREMENTS
- INVERT\_NEGS
- MATH
- NULL\_RETURNS
- PRIMITIVE\_RETURNS
- REMOVE\_CONDITIONALS\_EQUAL\_ELSE
- REMOVE\_CONDITIONALS\_ORDER\_ELSE
- TRUE\_RETURNS
- VOID\_METHOD\_CALLS

## Tests examined

- io.FileIOTest.[engine:junit-jupiter]/[class:io.FileIOTest]/[method:testFileDoesNotExist\_ThrowsException()] (1 ms)
- io.FileIOTest.[engine:junit-jupiter]/[class:io.FileIOTest]/[method:testValidFile\_AllValidLines()] (1 ms)
- io.FileIOTest.[engine:junit-jupiter]/[class:io.FileIOTest]/[method:testEmptyFile\_ThrowsException()] (3 ms)
- io.FileIOTest.[engine:junit-jupiter]/[class:io.FileIOTest]/[method:testInvalidFile\_MixedValidAndInvalidLines()] (17 ms)
- math.ArrayOperationsTest.[engine:junit-jupiter]/[class:math.ArrayOperationsTest]/[method:findPrimesInFile()] (3 ms)

Report generated by [PIT](#) 1.20.0

## 1.1 Why Mutation Score Is Low (Detailed Causes)

From the mutation section of the report:

### ● **Survived Mutant:**

- **Line 33:**

```
if (!file.exists())
```

- **Mutation:** removed conditional - replaced equality check with false
- **Result:** Code was run, but the mutation was not caught (survived)
- **Cause:** The test that triggers this path (`testFileDoesNotExist_ThrowsException`) likely did not assert the exception properly or missed an edge case.

● **No Coverage:**

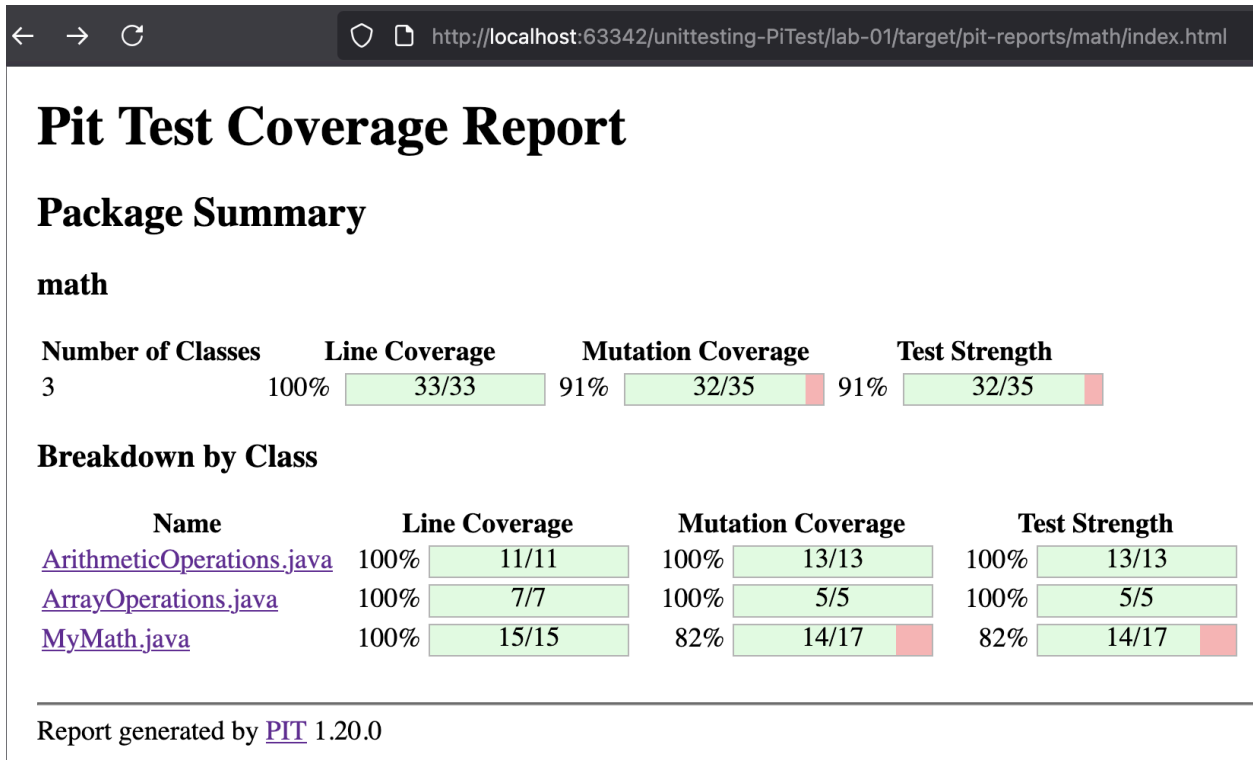
- **Line 50:**

```
e.printStackTrace();
```

- **Mutation:** removed call to `java/io/IOException::printStackTrace`
- **Result:** ☒ Not executed at all
- **Cause:** No test triggered the `IOException` block—no coverage here. This code is inside the outer `catch (IOException)` clause which is only hit if `new BufferedReader` or `readLine()` throws an I/O error.



## 2. For io package, MyMath.java class shows the low score



## MyMath.java

```
1 package math;
2
3 /**
4  * The MyMath provides simple methods such as computing a factorial or finding
5  * whether an integer is prime
6  *
7  * @author pandeliskirpoglou
8  * @version 1.0
9  * @since 2020-04-15
10 */
11
12 public class MyMath {
13
14     /**
15      * Gets one integer and returns it's factorial.
16      *
17      * @param n the number of which we want the factorial
18      * @return fact the factorial of the number n
19      * @exception IllegalArgumentException when inputs n < 0 and n > 12
20      */
21
22     public int factorial(int n) {
23         int fact = 1;
24         if (n < 0 || n > 12) {
25             throw new IllegalArgumentException("number should be 0 or above and 12 or below");
26         } else {
27             for (int i = 1; i <= n; i++) {
28                 fact = fact * i;
29             }
30         }
31
32         return fact;
33     }
34
35     /**
36      * Gets one integer and returns true if it is prime and false if it is not.
37      *
38      * @param n the number we are trying to find out whether it is prime or not
39      * @return isPrimeNumber true if n is prime | false if n is not prime
40      * @exception IllegalArgumentException when inputs n < 2
41      */
42
43     public boolean isPrime(int n) {
44         boolean isPrimeNumber = true;
45         if (n < 2) {
46             throw new IllegalArgumentException("No prime numbers below 2");
47         } else {
48             for (int i = 2; i <= n / 2; ++i) { // Checking to n/2 for complexity
49                 if (n % i == 0) {
50                     isPrimeNumber = false;
51                     break;
52                 }
53             }
54         }
55
56         return isPrimeNumber;
57     }
58 }
59
```

## Mutations

1. removed conditional - replaced comparison check with false → KILLED  
2. removed conditional - replaced comparison check with false → KILLED  
3. changed conditional boundary → SURVIVED [Covering tests](#)

Covered by tests:

24

- math.MyMathTest.[engine:junit-jupiter]/[class:math.MyMathTest]/[method:factorialOutsideRange()]
- math.MyMathTest.[engine:junit-jupiter]/[class:math.MyMathTest]/[method:factorialWithinRange()]

4. changed conditional boundary → SURVIVED [Covering tests](#)

Covered by tests:

27

- math.MyMathTest.[engine:junit-jupiter]/[class:math.MyMathTest]/[method:factorialOutsideRange()]
- math.MyMathTest.[engine:junit-jupiter]/[class:math.MyMathTest]/[method:factorialWithinRange()]

281. changed conditional boundary → KILLED  
2. removed conditional - replaced comparison check with false → KILLED

281. Replaced integer multiplication with division → KILLED

321. replaced int return with 0 for math/MyMath::factorial → KILLED

451. changed conditional boundary → KILLED

2. removed conditional - replaced comparison check with false → KILLED

1. removed conditional - replaced comparison check with false → KILLED

2. changed conditional boundary → SURVIVED [Covering tests](#)

Covered by tests:

48

- math.MyMathTest.[engine:junit-jupiter]/[class:math.MyMathTest]/[method:isPrime()]
- math.ArrayOperationsTest.[engine:junit-jupiter]/[class:math.ArrayOperationsTest]/[method:findPrimesInFile()]

3. Replaced integer division with multiplication → KILLED

491. Replaced integer modulus with multiplication → KILLED

2. removed conditional - replaced equality check with false → KILLED

561. replaced boolean return with true for math/MyMath::isPrime → KILLED

2. replaced boolean return with false for math/MyMath::isPrime → KILLED

## Active mutators

- CONDITIONALS\_BOUNDARY
- EMPTY\_RETURNS
- FALSE\_RETURNS
- INCREMENTS
- INVERT\_NEGS
- MATH
- NULL\_RETURNS
- PRIMITIVE\_RETURNS
- REMOVE\_CONDITIONALS\_EQUAL\_ELSE
- REMOVE\_CONDITIONALS\_ORDER\_ELSE
- TRUE\_RETURNS
- VOID\_METHOD\_CALLS

## Tests examined

- math.MyMathTest.[engine:junit-jupiter]/[class:math.MyMathTest]/[method:isPrime()] (0 ms)
- math.ArrayOperationsTest.[engine:junit-jupiter]/[class:math.ArrayOperationsTest]/[method:findPrimesInFile()] (3 ms)
- math.MyMathTest.[engine:junit-jupiter]/[class:math.MyMathTest]/[method:factorialWithinRange()] (0 ms)
- math.MyMathTest.[engine:junit-jupiter]/[class:math.MyMathTest]/[method:factorialOutsideRange()] (0 ms)
- math.MyMathTest.[engine:junit-jupiter]/[class:math.MyMathTest]/[method:isPrimeLessThanTwo()] (1 ms)

## 2.1 Why Mutation Score Is Low (Detailed Causes)

### 💣 Survived Mutants (Weak Areas)

From the mutation report and code:

#### ● 1. Line 24 — Input validation condition

```
if (n < 0 || n > 12)
```

- **Mutation:** changed conditional boundary — `n > 12` changed to `n >= 12`
- **Status:** SURVIVED
- **Root Cause:**
  - The test does not check the exact boundary `n = 12`, which would differentiate between `>` and `>=`.
  - Only checks for values like `n = 13`, missing edge case.
  - Add test: `factorial(12)` — it should succeed. If it fails, that would catch the mutant.

## ● 2. Line 45 — Prime check loop condition

```
for (int i = 2; i <= n / 2; ++i)
```

**Mutation:** changed loop boundary ( $\leq$  to  $<$ ) or altered range

**Status:** SURVIVED

**Root Cause:**

- The test does not check composite numbers with **only one divisor in the upper half**.
- e.g.,  $n = 4 \rightarrow$  only divisible by 2. If the loop range is off by one, it may miss this.
- Add test: `isPrime(4)`  $\rightarrow$  expected result: `false`.

## Step-3: A detailed analysis of the factors that led to performance improvements in low-scoring classes.

### 3.1 After adding test cases, Overall results:



http://localhost:63342/unittesting-PiTest/lab-01/target/pit-

## Pit Test Coverage Report

### Project Summary

| Number of Classes | Line Coverage         | Mutation Coverage     | Test Strength         |
|-------------------|-----------------------|-----------------------|-----------------------|
| 4                 | 100% <div>51/51</div> | 100% <div>40/40</div> | 100% <div>40/40</div> |

### Breakdown by Package

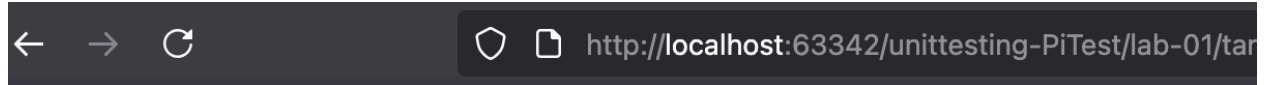
| Name                 | Number of Classes | Line Coverage         | Mutation Coverage     | Test Strength         |
|----------------------|-------------------|-----------------------|-----------------------|-----------------------|
| <a href="#">io</a>   | 1                 | 100% <div>18/18</div> | 100% <div>5/5</div>   | 100% <div>5/5</div>   |
| <a href="#">math</a> | 3                 | 100% <div>33/33</div> | 100% <div>35/35</div> | 100% <div>35/35</div> |

---

Report generated by [PIT](#) 1.20.0

Enhanced functionality available at [arcmutate.com](http://arcmutate.com)

## 3.2 For io package, FileIOTest.java and FileIO.java improvement details below



# Pit Test Coverage Report

## Package Summary

io

| Number of Classes | Line Coverage         | Mutation Coverage   | Test Strength       |
|-------------------|-----------------------|---------------------|---------------------|
| 1                 | 100% <div>18/18</div> | 100% <div>5/5</div> | 100% <div>5/5</div> |

## Breakdown by Class

| Name                        | Line Coverage         | Mutation Coverage   | Test Strength       |
|-----------------------------|-----------------------|---------------------|---------------------|
| <a href="#">FileIO.java</a> | 100% <div>18/18</div> | 100% <div>5/5</div> | 100% <div>5/5</div> |

---

Report generated by [PIT](#) 1.20.0



http://localhost:63342/unittesting-PiTest/

## FileIO.java

```
1 package io;
2
3 import java.io.BufferedReader;
4 import java.io.File;
5 import java.io.FileNotFoundException;
6 import java.io.FileReader;
7 import java.io.IOException;
8 import java.util.ArrayList;
9 import java.util.List;
10
11 /**
12  * The FileIO provides simple file input/output operations that serve as
13  * hands-on practice on Unit Testing.
14  *
15  * @author agkortzis
16  * @version 1.0
17  * @since 2020-04-06
18  */
19 public class FileIO {
20
21     /**
22      * Reads a file that contains numbers line by line
23      * and returns an array of the integers found in the file.
24      * @param filepath the file that contains the numbers
25      * @return an array of numbers
26      * @exception IllegalArgumentException when the given file does not exist
27      * @exception IllegalArgumentException when the given file is empty
28      * @exception NumberFormatException for checking invalid entries
29      * @exception IOException when an IO interruption occurs (not required to be tested)
30      */
31     public int[] readFile(String filepath) {
32         File file = new File(filepath);
33         if (!file.exists())
34             throw new IllegalArgumentException("Input file does not exist");
35
36         List<Integer> numbersList = new ArrayList<>();
37         BufferedReader reader;
38         try {
39             reader = new BufferedReader(new FileReader(file));
40             String line = null;
41             while ((line = reader.readLine()) != null) {
42                 try {
43                     int number = Integer.parseInt(line);
44                     numbersList.add(number);
45                 } catch (NumberFormatException e) {
46                     // Do nothing will skip the the current invalid line
47                 }
48             }
49         } catch (IOException e) {
50             throw new RuntimeException("IO Error while reading the file", e);
51         }
52
53         if (numbersList.size() == 0)
54             throw new IllegalArgumentException("Given file is empty");
55
56         // Convert a List to an array using
57         return numbersList.stream().mapToInt(i -> i).toArray();
58     }
59
60 }
```



## Mutations

|    |                                                                                                                                                  |
|----|--------------------------------------------------------------------------------------------------------------------------------------------------|
| 33 | 1. removed conditional - replaced equality check with false → KILLED                                                                             |
| 41 | 1. removed conditional - replaced equality check with false → KILLED                                                                             |
| 53 | 1. removed conditional - replaced equality check with false → KILLED                                                                             |
| 57 | 1. replaced return value with null for io/FileIO::readFile → KILLED<br>2. replaced int return with 0 for io/FileIO::lambda\$readFile\$0 → KILLED |

## Active mutators

- CONDITIONALS\_BOUNDARY
- EMPTY\_RETURNS
- FALSE\_RETURNS
- INCREMENTS
- INVERT\_NEGS
- MATH
- NULL\_RETURNS
- PRIMITIVE\_RETURNS
- REMOVE\_CONDITIONALS\_EQUAL\_ELSE
- REMOVE\_CONDITIONALS\_ORDER\_ELSE
- TRUE\_RETURNS
- VOID\_METHOD\_CALLS

## Tests examined

- io.FileIOTest.[engine:junit-jupiter]/[class:io.FileIOTest]/[method:testValidFile\_AllValidLines()] (0 ms)
- io.FileIOTest.[engine:junit-jupiter]/[class:io.FileIOTest]/[method:testInvalidFile\_MixedValidAndInvalidLines()] (2 ms)
- math.ArrayOperationsTest.[engine:junit-jupiter]/[class:math.ArrayOperationsTest]/[method:findPrimesInFile()] (0 ms)
- io.FileIOTest.[engine:junit-jupiter]/[class:io.FileIOTest]/[method:testFileDoesNotExist\_ThrowsException\_WithMessage()] (0 ms)
- io.FileIOTest.[engine:junit-jupiter]/[class:io.FileIOTest]/[method:testEmptyFile\_ThrowsException()] (0 ms)
- io.FileIOTest.[engine:junit-jupiter]/[class:io.FileIOTest]/[method:testFileWithOnlyInvalidLines\_ThrowsException()] (1 ms)
- io.FileIOTest.[engine:junit-jupiter]/[class:io.FileIOTest]/[method:testIOExceptionWithDirectoryInsteadOfFile()] (2 ms)
- io.FileIOTest.[engine:junit-jupiter]/[class:io.FileIOTest]/[method:testIOExceptionHandling()] (14 ms)

```

1 package io;
2
3 import org.junit.jupiter.api.AfterEach;
4 import org.junit.jupiter.api.BeforeEach;
5 import org.junit.jupiter.api.Test;
6
7 import java.net.URISyntaxException;
8 import java.nio.file.Path;
9 import java.nio.file.Paths;
10
11 import static org.junit.jupiter.api.Assertions.*;
12
13 class FileIOTest {
14
15     private FileIO fileIO; 8 usages
16
17     @BeforeEach
18     void setUp() {
19         fileIO = new FileIO();
20     }
21
22     @AfterEach
23     void tearDown() {
24         fileIO = null;
25     }
26
27     private String getPath(String resourceName) { 5 usages
28         try {
29             return Paths.get(getClass().getClassLoader().getResource(resourceName).toURI()).toString();
30         } catch (URISyntaxException | NullPointerException e) {
31             throw new IllegalArgumentException("Resource not found: " + resourceName, e);
32         }
33     }
34

```

```

13      class FileIOTest {
14
15          // Existing test case-1 for a file with all valid lines
16          @Test
17          void testValidFile_AllValidLines() {
18              String filePath = getPath( resourceName: "grades_valid.txt");
19              int[] expected = {3, 9, 0, 2, 10, 9, 3, 8, 0, 3};
20              int[] result = fileIO.readFile(filePath);
21              assertEquals(expected, result);
22          }
23
24          // Existing test case-2 for a file with mixed valid and invalid lines
25          @Test
26          void testInvalidFile_MixedValidAndInvalidLines() {
27              String filePath = getPath( resourceName: "grades_invalid.txt");
28              int[] expected = {3, 9, 2, 10, 8, 0, 3};
29              int[] result = fileIO.readFile(filePath);
30              assertEquals(expected, result);
31          }
32
33          // Existing test case-3 for a file with only invalid lines
34          @Test
35          void testFileWithOnlyInvalidLines_ThrowsException() {
36              String filePath = getPath( resourceName: "only_invalid.txt");
37              assertThrows(IllegalArgumentException.class, () -> fileIO.readFile(filePath));
38          }
39
40          // Existing test case-4 for an empty file
41          @Test
42          void testEmptyFile_ThrowsException() {
43              String filePath = getPath( resourceName: "empty_file.txt");
44              assertThrows(IllegalArgumentException.class, () -> fileIO.readFile(filePath));
45          }
46
47      }

```

```

67      // Existing test case-5 for a file does not exist
68      @Test
69      void testFileDoesNotExist_ThrowsException_WithMessage() {
70          String filePath = "nonexistent.txt";
71          Exception exception = assertThrows(IllegalArgumentException.class, () -> fileIO.readFile(filePath));
72          assertTrue(exception.getMessage().contains("Input file does not exist"));
73      }
74
75
76      // New test case-1: IOException when trying to read a directory instead of a file
77      @Test
78      void testIOExceptionWithDirectoryInsteadOfFile() {
79          String directoryPath = getPath( resourceName: ""); // Should resolve to /resources or base dir
80          Exception exception = assertThrows(RuntimeException.class, () -> fileIO.readFile(directoryPath));
81          assertTrue(exception.getMessage().contains("IO Error"));
82      }
83
84  }

```

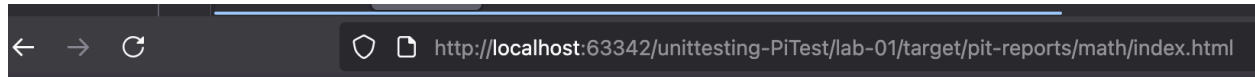
And in FileIO.java class, i change a little bit

```

49      } catch (IOException e) {
50          // e.printStackTrace(); // Uncomment this line to print the stack trace and add below line to throw an exception
51          throw new RuntimeException("IO Error while reading the file", e);
52      }

```

### 3.3 For math package, MyMathTest.java and MyMath.java improvement details below



## Pit Test Coverage Report

### Package Summary

math

| Number of Classes | Line Coverage         | Mutation Coverage     | Test Strength         |
|-------------------|-----------------------|-----------------------|-----------------------|
| 3                 | 100% <div>33/33</div> | 100% <div>35/35</div> | 100% <div>35/35</div> |

### Breakdown by Class

| Name                                      | Line Coverage         | Mutation Coverage     | Test Strength         |
|-------------------------------------------|-----------------------|-----------------------|-----------------------|
| <a href="#">ArithmeticOperations.java</a> | 100% <div>11/11</div> | 100% <div>13/13</div> | 100% <div>13/13</div> |
| <a href="#">ArrayOperations.java</a>      | 100% <div>7/7</div>   | 100% <div>5/5</div>   | 100% <div>5/5</div>   |
| <a href="#">MyMath.java</a>               | 100% <div>15/15</div> | 100% <div>17/17</div> | 100% <div>17/17</div> |

---

Report generated by [PIT](#) 1.20.0

## MyMath.java

```
1 package math;
2
3 /**
4  * The MyMath provides simple methods such as computing a factorial or finding
5  * whether an integer is prime
6  *
7  * @author pandeliskirpoglou
8  * @version 1.0
9  * @since 2020-04-15
10 */
11
12 public class MyMath {
13
14     /**
15      * Gets one integer and returns it's factorial.
16      *
17      * @param n the number of which we want the factorial
18      * @return fact the factorial of the number n
19      * @exception IllegalArgumentException when inputs n < 0 and n > 12
20      */
21     public int factorial(int n) {
22         int fact = 1;
23         if (n < 0 || n > 12) {
24             throw new IllegalArgumentException("number should be 0 or above and 12 or below");
25         } else {
26             for (int i = 1; i <= n; i++) {
27                 fact = fact * i;
28             }
29         }
30     }
31
32     return fact;
33 }
34
35 /**
36  * Gets one integer and returns true if it is prime and false if it is not.
37  *
38  * @param n the number we are trying to find out whether it is prime or not
39  * @return isPrimeNumber true if n is prime | false if n is not prime
40  * @exception IllegalArgumentException when inputs n < 2
41  */
42     public boolean isPrime(int n) {
43         boolean isPrimeNumber = true;
44         if (n < 2) {
45             throw new IllegalArgumentException("No prime numbers below 2");
46         } else {
47             for (int i = 2; i <= n / 2; ++i) { // Checking to n/2 for complexity
48                 if (n % i == 0) {
49                     isPrimeNumber = false;
50                     break;
51                 }
52             }
53         }
54     }
55
56     return isPrimeNumber;
57 }
58
59 }
```

## Mutations

|    |                                                                         |
|----|-------------------------------------------------------------------------|
| 24 | 1. removed conditional - replaced comparison check with false → KILLED  |
|    | 2. removed conditional - replaced comparison check with false → KILLED  |
|    | 3. changed conditional boundary → KILLED                                |
|    | 4. changed conditional boundary → KILLED                                |
| 27 | 1. changed conditional boundary → KILLED                                |
|    | 2. removed conditional - replaced comparison check with false → KILLED  |
| 28 | 1. Replaced integer multiplication with division → KILLED               |
| 32 | 1. replaced int return with 0 for math/MyMath::factorial → KILLED       |
| 45 | 1. changed conditional boundary → KILLED                                |
|    | 2. removed conditional - replaced comparison check with false → KILLED  |
| 48 | 1. removed conditional - replaced comparison check with false → KILLED  |
|    | 2. changed conditional boundary → KILLED                                |
|    | 3. Replaced integer division with multiplication → KILLED               |
| 49 | 1. Replaced integer modulus with multiplication → KILLED                |
|    | 2. removed conditional - replaced equality check with false → KILLED    |
| 56 | 1. replaced boolean return with true for math/MyMath::isPrime → KILLED  |
|    | 2. replaced boolean return with false for math/MyMath::isPrime → KILLED |

## Active mutators

- CONDITIONALS\_BOUNDARY
- EMPTY\_RETURNS
- FALSE\_RETURNS
- INCREMENTS
- INVERT\_NEGS
- MATH
- NULL\_RETURNS
- PRIMITIVE\_RETURNS
- REMOVE\_CONDITIONALS\_EQUAL\_ELSE
- REMOVE\_CONDITIONALS\_ORDER\_ELSE
- TRUE\_RETURNS
- VOID\_METHOD\_CALLS

## Tests examined

- math.MyMathTest.[engine:junit-jupiter]/[class:math.MyMathTest]/[method:testIsPrimeWithNine()] (0 ms)
- math.MyMathTest.[engine:junit-jupiter]/[class:math.MyMathTest]/[method:testIsPrimeWithFour()] (0 ms)
- math.MyMathTest.[engine:junit-jupiter]/[class:math.MyMathTest]/[method:testIsPrimeWithEleven()] (0 ms)
- math.MyMathTest.[engine:junit-jupiter]/[class:math.MyMathTest]/[method:isPrime()] (0 ms)
- math.ArrayOperationsTest.[engine:junit-jupiter]/[class:math.ArrayOperationsTest]/[method:findPrimesInFile()] (0 ms)
- math.MyMathTest.[engine:junit-jupiter]/[class:math.MyMathTest]/[method:testIsPrimeWithTwo()] (0 ms)
- math.MyMathTest.[engine:junit-jupiter]/[class:math.MyMathTest]/[method:factorialUpperBoundary()] (0 ms)
- math.MyMathTest.[engine:junit-jupiter]/[class:math.MyMathTest]/[method:factorialWithinRange()] (0 ms)
- math.MyMathTest.[engine:junit-jupiter]/[class:math.MyMathTest]/[method:factorialOfTwo()] (0 ms)
- math.MyMathTest.[engine:junit-jupiter]/[class:math.MyMathTest]/[method:factorialOfOne()] (0 ms)
- math.MyMathTest.[engine:junit-jupiter]/[class:math.MyMathTest]/[method:factorialOutsideRange()] (0 ms)
- math.MyMathTest.[engine:junit-jupiter]/[class:math.MyMathTest]/[method:factorialLowerBoundary()] (0 ms)
- math.MyMathTest.[engine:junit-jupiter]/[class:math.MyMathTest]/[method:factorialOfZero()] (0 ms)
- math.MyMathTest.[engine:junit-jupiter]/[class:math.MyMathTest]/[method:isPrimeLessThanTwo()] (0 ms)

```

1 package math;
2
3     import org.junit.jupiter.api.AfterEach;
4     import org.junit.jupiter.api.BeforeEach;
5     import org.junit.jupiter.api.Test;
6
7     import static org.junit.jupiter.api.Assertions.*;
8
9     class MyMathTest {
10
11         private MyMath myMath; 15 usages
12
13          @BeforeEach
14         void setUp() {
15             myMath = new MyMath();
16         }
17
18         @AfterEach
19         void tearDown() {
20             myMath = null;
21         }
22
23         //  Existing test case-1 for valid factorial
24         @Test
25         void factorialWithinRange() {
26             int expectedValue = 720;
27             int actualValue = myMath.factorial(n: 6);
28             assertEquals(expectedValue, actualValue);
29         }
30
31         //  Existing test case-2 for upper-bound violation
32         @Test
33         void factorialOutsideRange() {
34             assertThrows(IllegalArgumentException.class,
35                 () -> myMath.factorial(n: 13));
36         }
37

```

```

9      class MyMathTest {
38          // ✓ Existing test case-3 for valid prime
39          @Test
40          ✓  void isPrime() {
41              boolean expectedValue = true;
42              boolean actualValue = myMath.isPrime( n: 7);
43              assertEquals(expectedValue, actualValue);
44          }
45
46          // ✓ Existing test case-4 for invalid prime input
47          @Test
48          ✓  void isPrimeLessThanTwo() {
49              assertThrows(IllegalArgumentException.class,
50                  () -> myMath.isPrime( n: -2));
51          }
52
53          // ▲ Additional tests for factorial edge cases
54          // Additional test case-1 for factorial of zero
55          @Test
56          ✓  void factorialOfZero() {
57              assertEquals( expected: 1, myMath.factorial( n: 0));
58          }
59
60          // Additional test case-2 for factorial of one
61          @Test
62          ✓  void factorialOfOne() {
63              assertEquals( expected: 1, myMath.factorial( n: 1));
64          }
65
66          // Additional test case-3 for factorial of twelve (upper boundary)
67          @Test
68          ✓  void factorialUpperBoundary() {
69              assertEquals( expected: 479001600, myMath.factorial( n: 12));
70          }
71      }

```



```
9      class MyMathTest {
72          // Additional test case-4 for factorial of negative number (lower boundary)
73          @Test
74          void factorialLowerBoundary() {
75              assertThrows(IllegalArgumentException.class,
76                  () -> myMath.factorial(n: -1));
77          }
78
79          // Additional test case-5 for factorial of two
80          @Test
81          void factorialOfTwo() {
82              assertEquals(expected: 2, myMath.factorial(n: 2));
83          }
84
85          > //...
86          @Test
87          void testIsPrimeWithTwo() {
88              assertTrue(myMath.isPrime(n: 2));
89          }
90
91
92          // Additional test case-2 for isPrime with one
93          @Test
94          void testIsPrimeWithFour() {
95              assertFalse(myMath.isPrime(n: 4));
96          }
97
98          // Additional test case-3 for isPrime with zero
99          @Test
100         void testIsPrimeWithNine() {
101             assertFalse(myMath.isPrime(n: 9));
102         }
103
104         // Additional test case-4 for isPrime with negative number
105         @Test
106         void testIsPrimeWithEleven() {
107             assertTrue(myMath.isPrime(n: 11));
108         }
```